Code & Methodology:

Code Design:

Generally this code was separated into computational and plotting functions, which could then be used to test different situations by changing input parameters, keeping it flexible and relatively user-friendly. We decided to make an N-dimensional atmosphere model with variable emissivitiy. The Earth's emissivity in the long wave is assumed to be total (1). Additionally, we've used an albedo of 0.3, the average of Earth from NASA's Earth Fact Sheet, unless otherwise stated.

Modeling Functions:

Two functions were used in modeling the atmospheric energy balance: a coefficient matrix creation function and an atmospheric modeling function. Fluxes for each layer are calculated using Stefan-Boltzmann, excluding emissivity: $F_i = \sigma * T_i^4$. The emissivities for each layer are included in the coefficient matrix, not used to convert from flux to temperature.

The coefficient function is the workhorse that makes an N-layer coefficient matrix taking number of layers (N) and the emissivity (ε) of each layer as inputs. This function returns an N+1 square matrix which is symmetric along the diagonal. The first row represents the surface ('0th' Layer) and the final row represents the top-of-the-atmosphere layer (Nth Layer).
To create this coefficient matrix the following steps are performed:

1. First value in matrix is set at -1 (fixed pattern)
2. The surface row is then created using the pattern: $\epsilon * (1-\epsilon)^{\max(0,\text{col}-1)}$
3. Loop from 0 to N+1, creating diagonals based on the surface row, add them to a list of diagonals
4. Create the final coefficient matrix from the first diagonal entry (center diag)
5. Loop over the list of diagonals, placing them into the correct position by adding matrices.
6. Return the final matrix

This creation method is also described more in the answer to question 1 in the question-answering section.

The atmosphere modeling function takes in the number of layers (N), solar flux, emissivity (ε), albedo, and an optional value of 'short-wave absorbing layer', which defaults to 0 (surface). This function moves through the following sequence:

1. Matrix 'A' is created by calling the create coef function, taking the N, S_0, ε, and albedo.

2. Vector ‘b’ is made to be N+1 elements long, with all values being 0 except for the input short-wave absorbing layer. This value is set to be $\frac{-S_0(1-\alpha)}{4}$.
3. Next we try to solve for flux using np.linalg.solve(A, b). If this causes a singular matrix, we reduce the calculation to just be a no-atmosphere surface balance (first b value / first surface value)
4. We then loop over the fluxes to solve for temperature using $T_i = \sqrt[4]{F_i/\sigma}$.
5. Temperature list is returned, listed from the surface to the Nth layer.

Plotting Functions:

Two plotting functions are used to explore N-layer surface temperatures (fixed emissivity) and temperature profiles of an N-layer atmosphere.
Both functions take the following:

- number of layers (N)
- solar irradiance
- emissivity
- albedo
- temperature marker (optional)
    - marks a specific temperature w dotted line
- title (optional)
    - specify a specific title, defaults to basic thing
- short-wave absorbing layer (optional)
    - changes absorping layer from surface to other layer

This allows for quick changes of parameters to visualize changes in conditions.

Model Validation:

Validating this model used mainly the ‘DEBUG’ and ‘COEF_DEBUG’ features, printing additional information to ensure the code was working correctly.
To verify the coefficient and modelling functions, the output was compared to numerical values for a 1-layer atmosphere with emissivities of e = [0, 0.5, 1]. By using those emissivities we can verify everything is working because each case has a known solution. When e = 0, we expect the function and model to collapse to the standard energy balance with no atmosphere. Using an albedo of 0.3, we compared the output surface temperature to the NASA Earth Fact Sheet and saw it was the same (~254K). Additionally, the output coefficient matrix was what we would expect. Next, with e = 0.5 and e = 0.1, we can solve for the temperatures by hand, matching the output values with the hand-calculated values. Additionally, the output coefficent matrices matched the hand-built matricies done in class. As these worked as expected, the increased-layer atmospheres were also expected to work. These could also be checked using the

COEF_DEBUG option while running the code. This prints out the first row of coefficients and the full final coefficient matrix. The first row is printed in the $\epsilon * (1-\epsilon)^{\max(0,\text{col}-1)}$ style (ex. [-1, e*(1-e)^0, e*(1-e)^1, e*(1-e)^2, etc]) , which helps compare the output to the general form of **A**. Additionally, functions were tested to make sure that error catching worked. All of these are able to be checked by uncommenting code in the 'validation' section of the script.

Answers to Questions/Tasks:

Question/Task 1) Generate an N-Layer atmospheric energy balance model in matrix form $\boldsymbol{A}\vec{x} = \vec{b}$

An N-Layer energy balance model solved via matricies is symmetric along the center diagonal. If emissivity is 1, it reduces to a simple matrix. Otherwise, we can use the first row and create 'N' diagonals to construct the coefficient matrix and solve for the flux vector, **x.**

If emissivity = 1:

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b}, \boldsymbol{F_i} = \boldsymbol{\sigma T_i}^{\mathbf{4}}$$

$$A = \begin{bmatrix} -1 & 1 & 0 & \cdots & 0 & 0 \\ 1 & -2 & 1 & 0 & \ddots & 0 \\ 0 & 1 & -2 & 1 & 0 & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & \ddots & 0 & 1 & -2 & 1 \\ 0 & 0 & \ldots & 0 & 1 & -2 \end{bmatrix}$$

$$x = \begin{bmatrix} F_{surface} \\ \vdots \\ F_n \end{bmatrix} \quad b = \begin{bmatrix} -S/4\,(1-\alpha) \\ \vdots \\ 0 \end{bmatrix}$$

To solve for the **x**, we can multiply by the inverse of **A** on the left-hand side. This would have $\mathbf{x} = \mathbf{A}^{-1} * \mathbf{b}$. From there, we can go element-wise on **x** to solve for the temperatures at each value.

General form of **A:**

$$\mathrm{A}:\begin{bmatrix} -1 & \varepsilon(1-\varepsilon)^0 & \varepsilon(1-\varepsilon)^1 & \cdots & \varepsilon(1-\varepsilon)^{col-1} & \cdots & \varepsilon(1-\varepsilon)^{n-1} \\ \varepsilon(1-\varepsilon)^0 & -2\varepsilon & \varepsilon^2(1-\varepsilon)^0 & \varepsilon^2(1-\varepsilon)^1 & \ddots & \varepsilon * A_{[0,col-row]} & \vdots \\ \varepsilon(1-\varepsilon)^1 & \varepsilon^2(1-\varepsilon)^0 & -2\varepsilon & \varepsilon^2(1-\varepsilon)^0 & \varepsilon^2(1-\varepsilon)^1 & \ddots & \varepsilon * A_{[0,col-row]} \\ \vdots & \varepsilon^2(1-\varepsilon)^1 & \varepsilon^2(1-\varepsilon)^0 & \ddots & \varepsilon^2(1-\varepsilon)^0 & \varepsilon^2(1-\varepsilon)^1 & \vdots \\ \varepsilon(1-\varepsilon)^{row-1} & \ddots & \varepsilon^2(1-\varepsilon)^1 & \varepsilon^2(1-\varepsilon)^0 & -2\varepsilon & \varepsilon^2(1-\varepsilon)^0 & \varepsilon^2(1-\varepsilon)^1 \\ \vdots & \varepsilon * A_{[0,col-row]} & \ddots & \varepsilon^2(1-\varepsilon)^1 & \varepsilon^2(1-\varepsilon)^0 & -2\varepsilon & \varepsilon^2(1-\varepsilon)^0 \\ \varepsilon(1-\varepsilon)^{n-1} & \cdots & \varepsilon * A_{[0,col-row]} & \ldots & \varepsilon^2(1-\varepsilon)^1 & \varepsilon^2(1-\varepsilon)^0 & -2\varepsilon \end{bmatrix}$$

When $\varepsilon = 0$, this reduces to the matrix with -2 and 1 values along the diagonals. We can use this form to vary emissivity in the next section when developing a python modeling function.

Question/Task 2) Create a function to find the temperatures at each layer for an N-layer energy balance model.

This is completed in the accompanying python script using the “create coefficient matrix” and “model atmosphere” functions.

Question/Task 3) How does surface temperature of Earth depend on emissivity and number of atmospheric layers?

3a) For an average Earth surface temperature of 288K, what does your model predict for the emissivity of Earth's atmosphere?

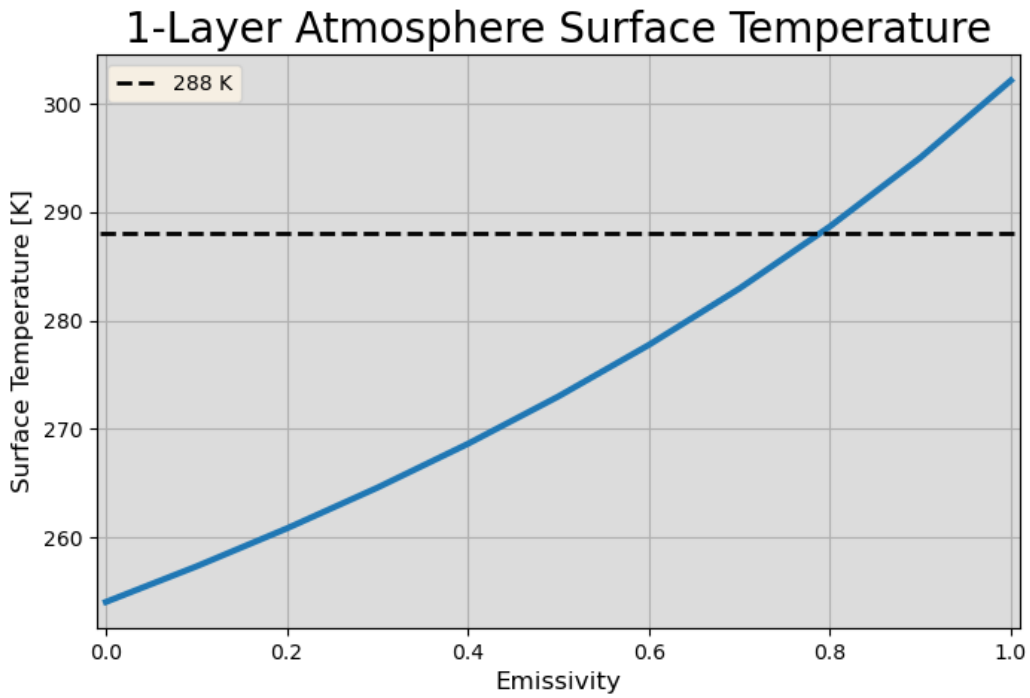


Figure 1. One-layer Earth atmosphere surface temperatures due to varying emissivity. Target temperature, 288K, is shown in dotted black line.

Using a single-layer atmosphere, we can see that surface temperature depends heavily on the value of emissivitiy. With the one-layer model, temperature increases with increasing emissivity, as expected. For the Earth itself, we expect an average emissivity of ~0.8, where the dotted line and temperature curves intersect. This is relatively high- much higher than the given value in the second part of this question, 0.255. When setting the albedo to 0, we get a closer value to that of Earth's average, which points to there needing to be a balance between layer amount and emissivity to get realistic surface temperatures.

3b) Using this emissivity value, how many layers of atmosphere are required to produce a surface temperature of ~288K?

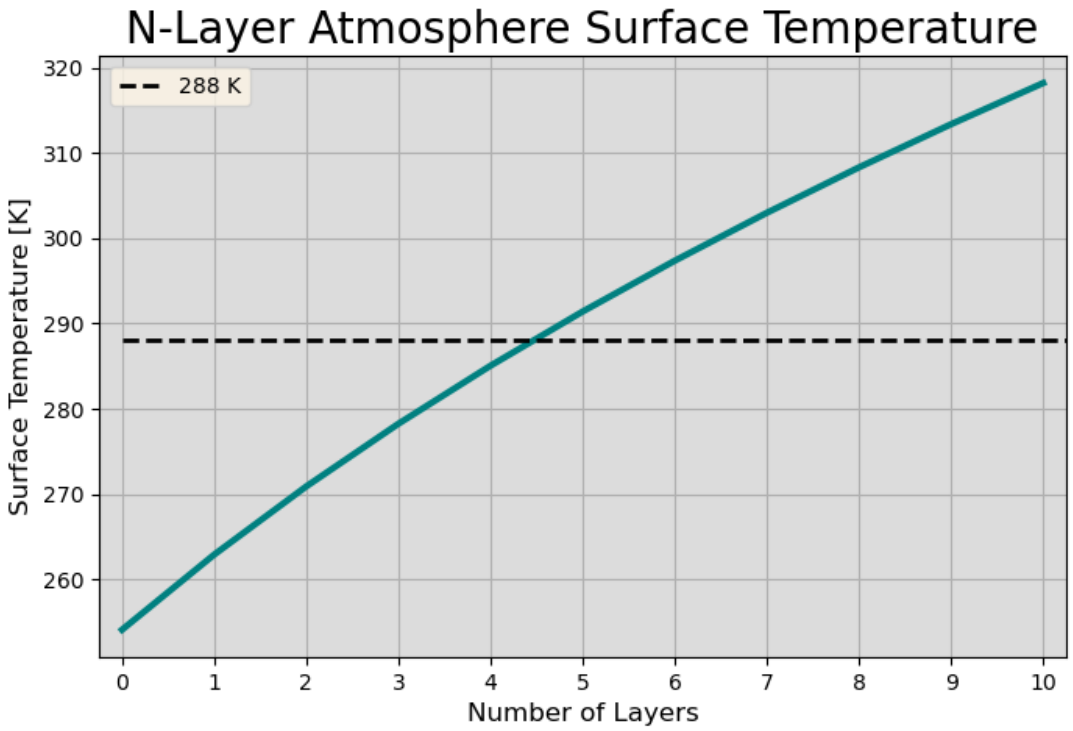


Figure 2. Earth energy balance model using N-Layer atmosphere vs surface temperature. Emissivity is average value (0.255) and dashed black line is the goal temperature, 288K.

We will need about four or five atmospheric layers to get a surface temperature of 288K. As shown in Figure 2, with an emissivity of 0.255, the temperature is 288K right between the 4th and 5th layer. Calculating the surface temperatures of the four and five-layer and finding the difference from 288K can help us select the best atmosphere to use. We find the following:

4-Layer Difference from 288K: 2.95K
5-Layer Difference from 288K: 3.40K

As such, we will use the 4-Layer Atmosphere as for our temperature profile, since it is closer to 288K than the 5-Layer.

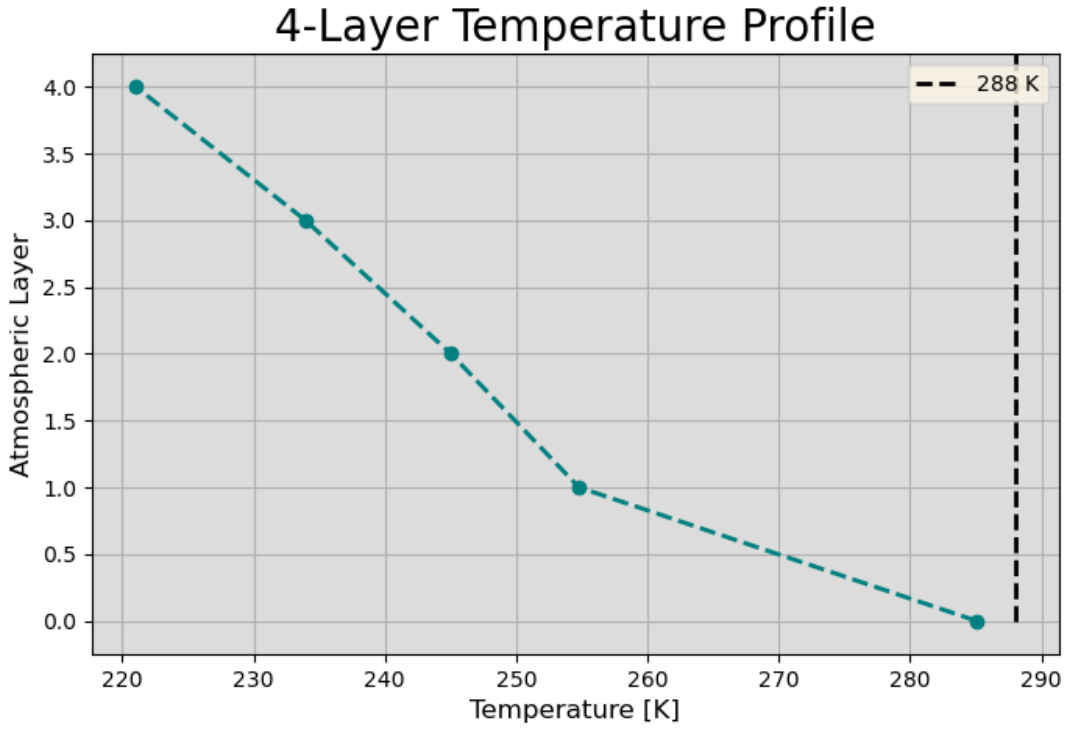


Figure 3. 4-Layer temperature profile for Earth energy atmosphere energy balance model.

Our four-layer temperature profile depicted in Figure 3 shows the decreasing temperature as a function of atmospheric layer, our stand-in for height. Our surface temperature, layer = 0, is slightly cooler than the 288K by ~2.95K (calculated above).

How does emissivity and number of layers affect the surface temperature of Earth?

Increasing emissivity causes more energy to be absorbed regardless of the number of layers. The number of layers then blankets the surface, increasing the temperature. With a given emissivity, increasing the number of layers will increase surface temperature. Similarly, with a given number of layers, increasing emissivity will increase surface temperature. For both of these the opposite is true as well. To have a surface temperature similar to the Earth, we could choose to have a low number of layers with an unrealistic emissivity (as see in Figure 1). Or, we could lower the emissivity to a more realistic value, 0.255, and increase the number of layers giving us a more realistic picture (Figure 2).

Question 4) How many atmospheric layers do we expect on the planet Venus, given that the surface temperature is ~700K.

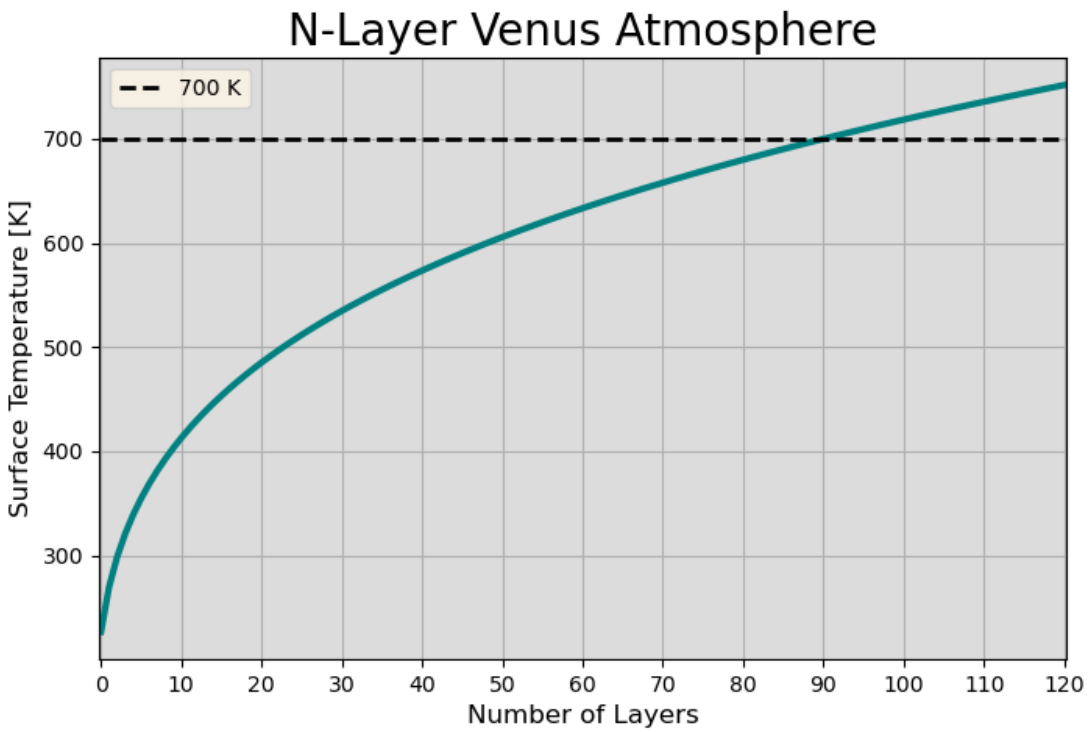


Figure 4. N-Layer Venus atmospheric energy balance. Emissivity is 1 and albedo is 0.7. The black dashed line represents the approximate acutal surface temperature, 700K.

We will need about 90 atmospheric layers to get Venus's surface temperature to ~700K. This is assuming that the albedo of Venus is very high (~0.77 as per the Venus Fact Sheet) and each layer absorbs perfectly (ε = 1). The dependance of surface temperature on the number of layers is shown in Figure 4.

Question 5) What would Earth's surface temperature be under a nuclear winter scenario?

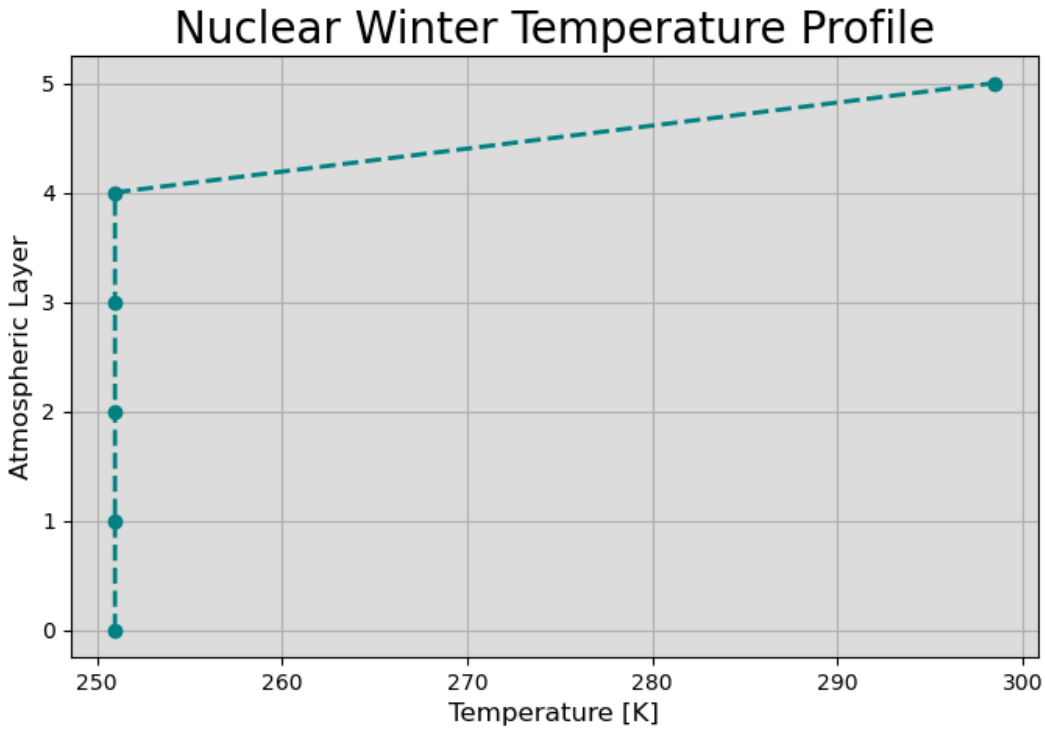


Figure 5. Earth energy balance under a 'nuclear winter scenario' where the highest atmospheric layer absorbs all longwave radiation, re-emitting with emissivity of 0.5 (same with each layer).

In this situation, the Earth's surface is very cold (~250K), while the top-layer is quite hot ~300K. The top layer of the atmosphere perfectly absorbs all the shortwave radiation, re-emitting worse in the longwave, with an emissivity of 0.5 for all atmospheric levels. This could be a poor approximation, with a better value being $\varepsilon = 1$ for the top level, which would warm the surface to have a less-frigid temperature. Otherwise, Figure 5 shows the temperature profile of the Earth in this situation, where the atmosphere itself is equally temperatured, being quite cold.